

Diagnose R9700 Mapping Issues

08/23/2026 19:36:18

<https://chatgpt.com/c/6a8b1b3a-b178-83ea-b486-083c67d02ab0>

I'll reconstruct what changed and test whether the new evidence truly...



I'll reconstruct what changed and test whether the new evidence truly separates queue/cache state from kernel arithmetic, while validating key assumptions against TinyGPU and upstream AMD paths.

The new fault capture is valuable, but I'm checking a potentially decisive interpretation error: `GCMV_L2_PROTECTION_FAULT_ADDR` appears to identify a logical page, not the MQD physical page.

The cache and RSRC3 tests are useful eliminations; next I want same-path sentinel and pre/post-fault controls to establish whether the apparent CPF mapping fault belongs to this dispatch.

Verdict

You are moving in the right general direction, and the new fault-register capture is the most valuable evidence gathered so far. However, the transcript's final conclusion is too strong and partly incorrect.

The work now establishes that:

- The broader `ACQUIRE_MEM` cache invalidation does **not by itself** make the epsilon kernel retire.
- Overriding `COMPUTE_PGM_RSRC3` to both `0` and `160` does **not by itself** change the timeout.
- The failing run exposes a GFXHUB fault from **CPF under VMID 0**, reading GPU virtual page `0x2003`, which corresponds to GPU virtual address `0x02003000`.

That last result moves the immediate blocker away from RMSNorm arithmetic and toward **command-processor address domains, MQD addressing, VMID 0 mappings, or stale queue/fault state**.

The important correction is:

`0x02003000` is reported as a **GPU virtual address**, not as the physical address of the page-table or MQD page.

Its numerical equality to your `kMqdPAddr` is highly suspicious, but it supports a different hypothesis: **a raw VRAM/BAR-relative physical offset may be getting**

consumed as a GPU-visible virtual address.

What the latest work successfully established

1. The cache-invalidation experiment was worthwhile

You added GLI, GL1, GL2, GLK, GLM, and GLV invalidation/writeback controls, rebuilt successfully, and observed:

- the `hidden` stage still returned 2,048 finite values with its expected digest;
- the epsilon candidate still timed out in `resident_dispatch`.

That tells us the additional cache operation is not a sufficient fix for this fault. `native-r9700-session-full-3.jsonl`

Keep the code for now as diagnostic hardening. Later, it should probably be scoped to **program installation or replacement**, rather than paying for a global instruction/L2 invalidation before every dispatch.

It does **not**, however, fully rule out an upload/cache problem, because the supposed sentinel did not use the failing resident-image path.

2. The RSRC3 override was a good experiment

The epsilon image timed out with both:

```
NATIVE_RSRC3_OVERRIDE=0
NATIVE_RSRC3_OVERRIDE=160
```

with the same expected output and the same resident-dispatch timeout. `native-r9700-session-full-3.jsonl`

This justifies strongly deprioritizing `INST_PREF_SIZE` as the primary cause. On GFX12, bits 11:4 of `COMPUTE_PGM_RSRC3` are indeed `INST_PREF_SIZE`, in units of 128 bytes. (LLVM)

I would phrase the result as:

RSRC3 is not sufficient to explain or repair the failure.

I would not yet say it has been absolutely ruled out, because the two override runs were performed sequentially without demonstrating a complete queue/fault reset

between them.

3. Adding the GCVM fault registers was exactly the right move

The new capture reports:

```
CP_MEC_RS64_EXCEPTION_STATUS = 0x0000c67a

GCVM_L2_PROTECTION_FAULT_STATUS_L032 = 0x00000933
GCVM_L2_PROTECTION_FAULT_STATUS_HI32 = 0x00000000

GCVM_L2_PROTECTION_FAULT_ADDR_L032 = 0x00002003
GCVM_L2_PROTECTION_FAULT_ADDR_HI32 = 0x00000000
```

native-r9700-session-full-3.jsonl

This is much more diagnostic than the earlier RS64 register alone.

Critical correction: the status was decoded incorrectly

The generated register definition used in the transcript assigns:

```
bit 0      MORE_FAULTS
bits 1:3    WALKER_ERROR
bits 4:7    PERMISSION_FAULTS
bit 8       MAPPING_ERROR
bits 9:17   CID
bit 18      RW
bits 20:23  VMID
```

native-r9700-session-full-3.jsonl

For 0x00000933, the correct extraction is:

Field	Calculation	Value
MORE_FAULTS	0x933 & 1	1
WALKER_ERROR	(0x933 >> 1) & 0x7	1
PERMISSION_FAULTS	(0x933 >> 4) & 0xf	3

Field	Calculation	Value
MAPPING_ERROR	$(0x933 \gg 8) \& 1$	1
CID	$(0x933 \gg 9) \& 0x1ff$	4
RW	$(0x933 \gg 18) \& 1$	0
VMID	$(0x933 \gg 20) \& 0xf$	0

The transcript eventually reported `WALKER_ERROR=3`. That is a bit-order error; the value is **1**.

The useful semantic interpretation is therefore:

- more than one fault is pending or has been observed;
- a nonzero walker error code of **1** is present;
- permission-fault bits **0** and **1** are set;
- a mapping error is set;
- the requesting client is ID **4**;
- it was a read;
- it occurred under VMID 0.

Linux's GFX12 client table maps client ID **4** to **CPF - Command Processor Frontend**. ([GitHub](#))

Linux's AMDGPU debugging documentation explains that:

- VMID 0 faults are generally associated with the kernel driver or firmware rather than a user process;
- CPF is the Command Processor Frontend;
- permission value **3** means the PTE was invalid and its read permission was not set;
- `RW=0` is a read. ([GitHub](#))

This is much more consistent with a **queue or command-processor address problem** than a shader-side RMSNorm arithmetic fault.

`0x2003` is a GPU virtual page, not a physical MQD page

The transcript's concluding interpretation was:

fault page `0x2003` → byte address `0x2003000` → numerically equal to `kMqdPaddr` → therefore the walker faulted on the MQD physical page.

The first arithmetic step is right:

```
GPU virtual page = 0x2003
GPU virtual byte address = 0x2003 << 12
                        = 0x02003000
```

Linux's GFX12 fault handler constructs the fault address by shifting the reported page value left by 12 and describes the result as the address of the page that faulted.

([GitHub](#)) The AMDGPU debugging guide explicitly calls it the **GPU virtual address that caused the fault**. ([GitHub](#))

Therefore:

```
0x02003000 is the faulting GPU virtual address.
```

It is **not evidence that the page-table walker was physically reading address `0x02003000`**. It is also not evidence that the walker read past the PTB or treated the MQD as a page table.

Your code does place `kMqdPaddr` at:

```
am_vm::kPtableArenaBase + 3 * 4096
```

which numerically yields `0x02003000`. `native-r9700-session-full.jsonl`

That numerical collision is very important-but the likely interpretation is:

Something may be programming the raw MQD physical/BAR offset `0x02003000` into a field that CPF later interprets as a GPU virtual address.

That is a substantially different root-cause hypothesis.

The strongest current hypothesis: an MQD address-domain mismatch

Linux's GFX12 initialization programs the MQD base from:

```
prop->mqd_gpu_addr
```

and describes the field as the pointer to the MQD:

```
mqd->cp_mqd_base_addr_lo = prop->mqd_gpu_addr & 0xffffffffc;  
mqd->cp_mqd_base_addr_hi = upper_32_bits(prop->mqd_gpu_addr);
```

The ring base is separately generated from `mqd_base_gpu_addr >> 8`. ([GitHub](#))

That suggests a high-value audit:

CP_MQD_BASE_ADDR must receive the address in the domain expected by CPF:
a GPU-visible/MC address, not necessarily a raw BAR0-relative VRAM offset.

If your native implementation currently programs:

```
CP_MQD_BASE_ADDR = kMqdPaddr = 0x02003000
```

and VMID 0 does not map low GPU VA `0x02003000`, then the new fault tuple fits remarkably well:

```
client      = CPF  
VMID        = 0  
operation   = read  
GPU VA      = 0x02003000  
mapping error = 1
```

This is now my leading technical hypothesis, **provided the fault was generated by the current candidate and was not stale**.

The value that TinyGPU/tinygrad calls `mqd_mc`, the value Linux calls `mqd_gpu_addr`, the raw VRAM/BAR offset, and the high GPU VA alias must be printed side by side. They should no longer be treated as interchangeable until proven equivalent.

Two experimental controls are still missing

1. The `hidden` stage is not a valid resident sentinel

The transcript's comparison later printed:

EMBED:

```
hsa_image_sha256: not_dispatched
kernarg_hex: not_dispatched
```

EPSILON:

```
stage: normalized
rmsnorm_kernel: llama_rmsnorm_epsilon_arithmetic_f16
```

`native-r9700-session-full-3.jsonl`

That means the successful `hidden` stage did not exercise the same resident HSA image-dispatch path as the epsilon candidate.

Therefore this conclusion is not established:

The hidden stage passed, so the resident queue and VM path were healthy immediately before epsilon.

Your sentinel must go through the exact same machinery:

- `ResidentHsaSession`;
- image allocation and upload;
- GPUVM mappings;
- `COMPUTE_PGM_*` programming;
- the same PM4 builder;
- the same queue;
- the same timeline completion path.

Use a resident constant-store or zero-store kernel, or the original resident RMSNorm image that is known to retire even though its result is numerically wrong.

The sentinel result should contain a real image digest, real kernargs, real resident GPU VA, and a successful resident fence.

2. The fault registers need a pre-dispatch baseline

You captured the fault after the candidate, but not before it. The status has `MORE_FAULTS=1`, and the same RS64 exception value has appeared across multiple failed runs. That leaves open the possibility that some or all of the captured state was sticky or inherited from a previous fault.

Before attributing `0x02003000` to epsilon or rsqrt:

1. Start from a cold or demonstrably reset device.
2. Read all fault registers before queue initialization.
3. Clear the fault state using a source-grounded mechanism.
4. Read it again to verify the clear.
5. Run exactly one resident sentinel.
6. Read it again.
7. Run exactly one candidate.
8. Read it again.
9. Do not run another candidate until the HQD/MEC reset is complete.

Until this is done, “the epsilon kernel caused the CPF fault at `0x02003000`” remains probable but unproven.

Recommended next debugging sequence

1. Correct the recorded conclusion

Keep commit `7abca9f`’s register instrumentation, but add a follow-up correction to its interpretation:

Correct:

- `WALKER_ERROR` = 1, not 3.
- `CID 4` = CPF.
- `VMID` = 0.
- `RW` = read.
- reported GPU VA = `0x02003000`.
- its numerical equality to `kMqdPaddr` suggests an address-domain mismatch.

Not yet established:

- that the fault address is physical;

- that the page-table walker accessed the MQD as a page-table page;
- that the candidate generated the fault rather than inheriting stale state.

2. Build a true resident sentinel sandwich

For each candidate:

```
full reset
pre-fault snapshot
resident sentinel
post-sentinel snapshot
candidate
post-candidate snapshot
attempt resident sentinel again
full reset
```

The post-candidate sentinel answers whether the candidate damaged queue state.

3. Log every relevant MQD address representation

At queue setup, print:

```
MQD BAR0/VRAM offset
MQD physical address used by your allocator
VRAM framebuffer base
VRAM MC offset/base
MQD MC address
MQD GPU virtual address
live CP_MQD_BASE_ADDR
live CP_MQD_BASE_ADDR_HI
CP_MQD_CONTROL.VMID
CP_HQD_VMID
VMID0 page-table root
```

Also print the corresponding tinygrad values from the successful path:

```
mqd_mc
ring_addr
rptr_addr
wptr_addr
```

The key equality to test is not:

```
CP_MQD_BASE_ADDR == raw physical offset
```

but:

```
CP_MQD_BASE_ADDR == address CPF expects in its configured VM/address domain
```

4. Run a relocation canary

Move the MQD physical allocation from `0x02003000` to an unmistakably different location, for example a page whose low digits cannot be confused with the existing PT arena.

Then rerun exactly one clean resident candidate.

Interpretation:

Result	Meaning
Fault GPU VA follows the new raw MQD physical offset	Strong proof that a physical offset is being consumed as a GPU VA
Fault remains at <code>0x02003000</code>	Stale fault, hard-coded address, or a different CP field
Fault becomes the intended high MQD GPU VA	Address conversion has changed, but mapping/PTE still needs repair
Fault disappears	MQD location, overlap, or address-domain programming was causal

This is probably the single most decisive experiment now.

5. Try a GPU-VA alias for the MQD

As a diagnostic-not as the final fix-map a high GPU VA to the MQD physical page and program `CP_MQD_BASE_ADDR` with that high GPU VA, assuming the register semantics in the applicable GFX12 path confirm this.

An alternative diagnostic is temporarily mapping low GPU VA `0x02003000` to the MQD page. If doing that makes the fault disappear, it strongly confirms that CPF is

interpreting the raw offset as a VA. Do not keep that as the final architecture; it would merely conceal the domain error.

6. Dump the live VMID 0 context

Capture:

```
GCVM_CONTEXT0_PAGE_TABLE_BASE_ADDR_L032/HI32
GCVM_CONTEXT0_PAGE_TABLE_START_ADDR_L032/HI32
GCVM_CONTEXT0_PAGE_TABLE_END_ADDR_L032/HI32
GCVM_CONTEXT0_CNTL
page-table depth
page-table block size
FB location/base/offset
system aperture
AGP aperture
identity-aperture settings
```

Then walk GPU VA `0x02003000` using the **live VMID 0 root**, not an expected or reconstructed root.

Also walk:

- the resident code VA;
- the resident descriptor VA;
- kernargs;
- ring;
- timeline;
- RPTR/WPTR;
- the intended MQD GPU VA.

This will reveal whether the fixed C0 mappings and newer resident mappings are actually in one coherent VMID 0 hierarchy.

7. Finish the PM4 markers

`HQD_PQ_RPTR=0x31` is valuable, but it only proves pointer advancement. It does not prove that the dispatch packet successfully launched a wave.

Keep the A/B/C/D plan:

```
A: CP write before DISPATCH_DIRECT
B: CP write after DISPATCH_DIRECT
C: write after CS_PARTIAL_FLUSH
D: normal RELEASE_MEM completion
```

Put the marker page in BAR0-visible VRAM and annotate every one of the 59 ring dwords.

8. Reopen “output salvage”

The fault-register portion is implemented, but the transcript does not show actual timeout output salvage. The failed result still reports blank image metadata and `gpu_va: 0`, rather than:

```
number of modified output dwords
first modified offset
last modified offset
first 64 raw output bytes
whether expected f16 value 0x5cf1 appeared
whether the sentinel pattern remained intact
```

```
native-r9700-session-full-3.jsonl
```

Split that task into:

```
[X] Capture fault registers on timeout
[ ] Salvage candidate output on timeout
```

Updated hypothesis ranking

Hypothesis	Current assessment
Stale/un-cleared fault state contaminating runs	High-priority uncertainty
Raw MQD physical/BAR offset programmed where CPF expects GPU address	Leading hypothesis if the fault is fresh
VMID 0 page-table root does not map the address domain used by CPF	High

Hypothesis	Current assessment
Another CP-visible pointer contains a physical offset rather than GPU VA	Medium-high
Queue teardown leaves CPF/MEC state active after a fault	Medium-high
Instruction-cache coherency alone	Deprioritized; full invalidation was not sufficient
<code>COMPUTE_PGM_RSRC3.INST_PREF_SIZE</code>	Strongly deprioritized
Shader rsqrt/sqrt instruction directly causes this timeout	Low for this specific fault: client is CPF, not SQC/TCP
Original RMSNorm NaN	Still a separate numerical problem; defer until resident launch is deterministic

Bottom line

You are **closer**, and the fault-register instrumentation was the correct next move. The project should now pivot from “why do these generated shader instructions hang?” to:

Why is CPF under VMID 0 attempting to read GPU VA `0x02003000`, and is that value coming from a raw MQD physical offset placed into `CP_MQD_BASE_ADDR` or another queue pointer?

Prove that with a clean pre/post fault baseline, a true resident sentinel, and the MQD relocation canary before spending more time on RMSNorm microkernels.

Sources

[User Guide for AMDGPU Backend - LLVM](#) [llvm.org](#)

[raw.githubusercontent.com](#) [raw.githubusercontent.com](#)

[linux/Documentation/gpu/amdgpu/debugging.rst at master · torvalds/linux · GitHub](#) [github.com](#)